SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

Messify: Multi-Tenant Mess Management SaaS Platform

Document Version:	1.0.0 (Production Ready)
Target Audience:	Full-Stack Software Engineers / Product Architects
Architecture Style:	Multi-Tenant SaaS (Software as a Service) with Shared DB & Tenant Isolation
Date of Generation:	June 16, 2026

1. Executive Summary & Project Overview

Messify is a multi-tenant Software-as-a-Service (SaaS) platform designed to automate and streamline operations for student hostels, shared apartments, and communal dining facilities (locally termed as "Messes"). In shared living ecosystems, tracking daily meal counts, managing grocery (bazar) contributions, calculating fluctuating utility/meal rates, and generating fair monthly financial settlements is notoriously prone to manual error, dispute, and lack of transparency.

This platform digitizes the entire lifecycle. Individual messes can register as unique sub-organizations (tenants). Members can track their personal meal footprints, submit verified grocery expenses, analyze real-time ranking matrices of financial contributions, and seamlessly resolve outstanding balances at the end of each billing cycle (month).

2. System Architecture & SaaS Multi-Tenancy Strategy

To ensure scalability, commercial viability, and security, the system must be architected using a modern SaaS approach:

- **Multi-Tenancy Model:** Logical separation via a Shared Database with Tenant Isolation. Every single table in the relational schema (except global system logs/plans) must include a `tenant_id` (or `mess_id`) foreign key column. Every API query must filter strictly on this key.

- **Subdomain Routing (Optional but Preferred):** Individual messes register a custom workspace URL, e.g., `pioneermess.messify.com`, where the sub-domain maps programmatically to the internal `tenant_id`.
- **State Persistence & Real-time Nature:** The backend must provide a stateless RESTful or GraphQL API layer, accompanied by a relational database (PostgreSQL recommended) to enforce ACID compliance for balance ledgers.

3. User Roles & Identity Access Management (IAM)

Users within a specific tenant workspace are bound to one of two structural roles. The permissions are explicitly mapped below:

3.1. Member Role

A standard user within a mess. Their system capability is restricted to personal logs and view-only operational transparent metrics.

- **Meal Tracking:** View their own real-time and historical daily meal counts.
- **Grocery Expense Logging:** Create an entry tracking capital spent out-of-pocket on groceries ("Bazar"). This entry remains in a *Pending Verification* state until processed by a manager.
- **Financial Auditing:** Access the global monthly dashboard, due balances, and historical monthly structural reports.

3.2. Manager Role

The administrative authority for a specific tenant workspace. This role inherits all Member permissions plus configuration privileges.

- **System Configurations:** Define monthly accounting periods, adjust historical settings, and modify active system parameters.
- **Expense Auditing & Verification:** Approve or reject pending grocery entries submitted by members to protect the integrity of the ledger.
- **Account Closure & Financial Settlement:** Trigger the automated reconciliation engine at the end of the month to freeze logs, compute exact meal rates, shift dues, and generate downloadable analytics.

4. Detailed Functional Requirements & Calculations

4.1. Grocery (Bazar) Management Engine

Any authenticated member can log an expense detailing money spent buying fresh food/inventory for the kitchen. Each item requires a monetary value, datetime stamp, itemized description, and receipt image attachment (optional).

The backend engine sums these values continuously to establish a running metric of **Total Grocery Contributions per Member** (C_m).

4.2. Gamified Grocery Contribution Ranking Matrix

To prevent freeloading and naturally delegate responsibilities, the SaaS must generate an algorithmic ascending ranking list. Members who have contributed the **least capital absolute amount** appear automatically at the very top of this interface.

Example Case Study Rendering:

Rank Position	Member Identity	Total Contribution (Tk)	Operational Directive / UI Indicator
1 (Critical Alert)	Suvo	500.00 Tk	Highest Priority for next Bazar run
2 (Warning)	Hasan	800.00 Tk	Medium Priority
3 (Compliant)	Rakib	1,200.00 Tk	Lowest Priority (Contributed Most)

4.3. Daily Meal Tracking Ledger

The system records absolute decimal or integer values for active meal intakes daily per user. A typical day supports fractions or numbers (e.g., Breakfast = 0.5, Lunch = 1.0, Dinner = 1.0, total 2.5 meals). For clarity, standard integer arrays can represent this layout:
• **Suvo:** 3 Meals | • **Rakib:** 2 Meals | • **Hasan:** 1 Meal.

4.4. Monthly Financial Deposit Vault

At the beginning of or during the billing cycle, members deposit lump-sum advance capital into the shared bank account or physical vault managed by the manager to clear advance liability.
• **Suvo:** 3,000.00 Tk | • **Rakib:** 3,000.00 Tk | • **Hasan:** 3,000.00 Tk.

5. Mathematical Core Engine & Financial Reconciliations

The backend engineer must implement a highly precise arithmetic engine utilizing fixed-point decimals (never floating-point types to prevent binary truncation errors). The computational workflow operates sequentially as follows:

Step 1: Universal Meal Rate Calculation

Executed dynamically or locked upon account closure. It maps the total verified system operational cost against the sum total of all consumed meals across all members.

$$\textit{Meal Rate } (R) = \textit{Total Monthly Expense } (E_{total}) \,/\, \textit{Total Meals Consumed } (M_{total})$$

Example Architecture Scenario: If $E_{total} = 30{,}000\ Tk$ and $M_{total} = 1{,}000$ meals, the calculated $R = 30.00\ Tk$ per meal.

Step 2: Individual Cost Attribution

Computes how much actual currency a singular member consumed out of the common resource pool based entirely on their physical consumption.

$$\textit{Member Cost } (C_m) = \textit{Individual Meal Count } (M_m) \times \textit{Meal Rate } (R)$$

Step 3: Final Balance Settlement Calculation

Calculates the net liquid financial positioning of the specific user at the precise moment of statement generation.

$$\textit{Final Balance } (B_{final}) = \textit{Individual Deposit } (D_m) - \textit{Member Cost } (C_m)$$

- If $B_{final} > 0$, the value represents a **Remaining Balance** / **Surplus Credit** to be rolled forward or refunded.
- If $B_{final} < 0$, the absolute value represents a **Due Amount** / **Payable Liability** that the member must deposit immediately to balance the books.

6. Database Schema Design (Entity Relationship Layout)

The developer must provision tables matching the normalized architectural specification below. All constraints and relations must be rigidly defined.

Table 1: Tenants (Messes)

Column Name	Data Type	Constraints	Description
`id`	UUID / BigInt	Primary Key	Unique identifier for the mess organization.
`name`	VARCHAR(150)	Not Null	The public name of the mess entity.
`subdomain`	VARCHAR(50)	Unique, Not Null	The routing prefix for multi-tenant isolation URL string.
`created_at`	Timestamp	Default NOW()	Audit logging tracing tenant instantiation.

Table 2: Users

Column Name	Data Type	Constraints	Description
`id`	UUID / BigInt	Primary Key	Unique identifier for the user.
`tenant_id`	UUID / BigInt	Foreign Key	References `Tenants.id` for strict data partitioning.
`email`	VARCHAR(255)	Unique, Not Null	User credential login index.
`role`	ENUM	'MEMBER', 'MANAGER'	Access control enforcement vector.

Table 3: Grocery_Expenses (Bazar Ledger)

Column Name	Data Type	Constraints	Description
`id`	UUID / BigInt	Primary Key	Unique transaction identifier.
`tenant_id`	UUID / BigInt	Foreign Key	References `Tenants.id`.
`user_id`	UUID / BigInt	Foreign Key	References `Users.id` (The purchaser).
`amount`	NUMERIC(12,2)	Not Null	Strict precision cost of the grocery procurement.
`status`	ENUM	'PENDING', 'VERIFIED'	Approval state controlled by the manager.

Table 4: Daily_Meals

Column Name	Data Type	Constraints	Description
`id`	UUID / BigInt	Primary Key	Unique record index.
`user_id`	UUID / BigInt	Foreign Key	References `Users.id`.
`date`	Date	Not Null	The calendar day for the tracking cycle.
`meal_count`	NUMERIC(4,2)	Default 0.00	Quantity of meals consumed (handles half meals).

7. API Specification (REST Endpoint Matrix)

The backend architecture must supply these clean JSON REST endpoints. All endpoints must parse authorization tokens containing security contexts to prevent cross-tenant parameter leakage.

7.1. Authentication & Scoped Identity

- `POST /api/v1/auth/register` - Initialize a new Tenant (Mess) and seed the initial Super Manager user.
- `POST /api/v1/auth/login` - Returns a JWT containing `tenant_id` and user `role` variables.

7.2. Grocery (Bazar) Services

- `POST /api/v1/groceries` - `Member` Authenticated member posts a new out-of-pocket grocery bill payload. Status defaults to `PENDING`.
- `GET /api/v1/groceries/ranking` - `All` Pulls structural collection sorted in `ASCENDING` order based on verified spend metrics.
- `PATCH /api/v1/groceries/:id/verify` - `Manager Only` Mutates status state machine to `VERIFIED`. Triggers transaction ledger inclusion.

7.3. Meal Tracking Services

- `POST /api/v1/meals/log-bulk` - `Manager Only` Batch updates daily multi-user matrix for meal intake values.
- `GET /api/v1/meals/personal` - `Member` Reads personal aggregated monthly meal footprint variables.

7.4. Account Closing & Reporting Services

- `POST /api/v1/reports/close-month` - `Manager Only` Executes final settlement calculations, snapshots structural analytical parameters, locks data modifications for the active billing cycle month, and flags alerts for negative due balances.
- `GET /api/v1/reports/summary` - `All` Pulls dynamic summaries containing total expenses, overall meals, dynamic meal rates, and explicit data detailing credit or due liabilities.

8. Non-Functional Requirements (SaaS Guardrails)

- **Security and Isolation:** Enforce database Row-Level Security (RLS) if utilizing PostgreSQL to hardcode logical bounds ensuring a user from *Tenant A* can never read or intercept network requests from *Tenant B*.

- **Data Precision:** Always map currency and numbers tracking meal points using arbitrary-precision types like `BigDecimal` (Java), `Decimal` (Python/C#), or PostgreSQL's native `NUMERIC` fields. Never utilize floats or doubles.
- **Scalable Analytics:** Index database foreign keys extensively (`tenant_id` , `user_id` , `date`) to allow immediate, unhindered compilation of real-time dashboards even when a specific tenant generates tens of thousands of historical logs.